

Úvod do databázových systémů

Kapitola 12

Normální formy relací – nástin

Motivace

- databázové schema (tj. struktura relací resp. tabulek a jejich vztahy) není pro konkrétní reálný problém dáno jednoznačně
 - tentýž problém lze řešit mnoha způsoby
 - jak poznat, které řešení je lepší nebo horší?
 - už z kapitoly o konceptuálním modelování víme, že entity (a odpovídající relace/tabulky) nemají být ani moc, ale ani málo obecné

Motivace

■ příklady

■ jeden extrém – příliš obecné entity

■ celá databáze uložena v jediné tabulce

- obrovská redundance dat
- špatná (vlastně žádná) struktura – nepřehledné (nevidíme detaily)
- nepružné (musím stále procházet i data, která nepotřebuji)
- vysoké riziko nekonzistence, obtížná kontrola integrity

■ druhý extrém – příliš specializované entity

■ mnoho tabulek o dvou sloupcích (klíč+atribut)

- rozdrobení dat, ztrácíme celkové vazby mezi daty
- nepřehledné (pro mnoho detailů nevidíme celek)
- pomalé (neustálé spojování mnoha tabulek)

Motivace

- optimum je někde mezi
 - pro každou úlohu může optimum ležet trochu jinde
 - chce to zkušenost a cit
 - lze nějak objektivně posoudit kvalitu návrhu?
 - ano – pomohou nám tzv. normální formy a normalizace relací
- normalizace relací
 - úprava struktury tak, aby relační schema dobře reprezentovalo data, aby byla omezena redundance a přitom se neztratily vazby mezi daty

Motivace

- příklad

- mějme relaci

PROGRAM(název_k, jméno_f, adresa, datum)

- problémy

- některá informace bude zbytečně opakována vícrát
 - adresa kina bude uložena tolikrát, kolik filmů kino hraje
 - změnil-li se jméno ulice, musíme přepsat mnoho řádků
 - pomalé
 - když některý řádek omylem vynecháme, bude databáze nekonzistentní – už nikdy se z ní s jistotou nedozvím, jak se ulice vlastně jmenuje
 - nehraje-li kino nic, ztrácíme jeho adresu
 - chceme-li přidat nové kino, lze to jen když se tam hraje nějaký film

Motivace

- jiný příklad
 - mějme relaci FILM, kde evidujeme
 - jméno filmu, herce, rok natočení
 - co když se rozhodneme evidovat u filmu víc herců?
 - **první řešení** – víc atributů herec

<u>NázevF</u>	Herec1	Herec2	Rok
Černí baroni	Vetchý	Landovský	1994
Kmotr	Brando		1972

- co když bude ale herců víc než dva?
 - nelze uložit
- co když bude herec jen jeden
 - zbytečná prázdná hodnota, která v klasickém RMD navíc není přípustná

Motivace

- jiný příklad
 - mějme relaci FILM, kde evidujeme
 - jméno filmu, herce, rok natočení
 - co když se rozhodneme evidovat u filmu víc herců?
 - **druhé řešení** – vícehodnotový atribut herec

<u>NázevF</u>	Herec	Rok
Černí baroni	Vetchý, Landovský	1994
Kmotr	Brando	1972

- v relačním modelu (a v řadě databázových strojů)
vícehodnotové atributy neexistují
 - není v první normální formě

Motivace

- jiný příklad

- mějme relaci FILM, kde evidujeme
 - jméno filmu, herce, rok natočení
- co když se rozhodneme evidovat u filmu víc herců?
- **třetí řešení** – více řádků pro jeden film

<u>NázevF</u>	<u>Herec</u>	Rok
Černí baroni	Vetchý	1994
Černí baroni	Landovský	1994

- redundance
 - aktualizací anomálie
- relační model neumožňuje první ani druhé řešení, třetí řešení je přitom také nevyhovující

Motivace

- problémy obecně

- redundance

- ukládání stejné informace víckrát na různých místech
 - zvyšuje prostorové nároky
 - riziko nekonzistence
 - pokud se změní jedna kopie redundantních dat, je třeba změnit i ostatní kopie, jinak se databáze stane nekonzistentní
 - obtížná kontrola integrity

- aktualizací anomálie (insert/delete/update)

- při vložení/mazání/aktualizaci dat příslušejících jednomu logickému objektu je potřeba zároveň vložit/smazat/upravit data náležející jinému objektu
 - např. když chci zapsat novou položku programu (co kde kdy se hraje), musím spolu s ní zapsat i adresu kina, což je zcela nesouvisející informace

Motivace

- řešení problému „PROGRAM“
 - dekompozice na dvě schemata relací
KINO(název_k, adresa)
PROGRAM(název_k, jméno_f, datum)
 - všechny problémy zmizely

- řešení problému „FILM“
 - dekompozice na tři schemata relací (vztah M:N)
HEREC(jméno_h, další atributy herce...)
FILM(jméno_f, rok)
HRAJE(jméno_h, jméno_f)
 - všechny problémy zmizely

Motivace

- co je důvodem problémů v příkladech?
 - hodnoty některých atributů závisejí na hodnotách jiných atributů
 1. ke každému kinu přísluší právě jedna adresa
 2. pro každé kino a film existuje nejvýše jedno datum, kdy se film dává
 - v podstatě jde o funkci: přiřazení hodnoty jiné hodnotě
 - speciální případ IO
- funkční závislosti
 - název_k $\rightarrow$ adresa
 - {název_k, jméno_f} $\rightarrow$ datum

Motivace

- co je důvodem problémů v příkladech?
 - hodnoty některých atributů závisejí na hodnotách jiných atributů
 1. ke každému filmu přísluší právě jeden rok natočení
- funkční závislosti
 - jméno_f $\rightarrow$ rok

Funkční závislosti

- závislost mezi dvěma množinami atributů v rámci jednoho schematu relace
 - závislost mezi daty (nikoli mezi entitami a daty ap.)
 - je to integritní omezení
 - vymezuje množinu přípustných relací
 - vzpomeňte na motivační příklad z přednášky o RMD

Funkční závislosti

- funkční závislost

- vyplývá z obecné (konceptuální) závislosti dat, nikoli z jejich konkrétní podoby

- např. máme tabulku studijních výsledků

<u>student</u>	<u>předmět</u>	známka
Novák	Teorie obvodů	3
Novák	Úvod do algebry	3
Haničinec	Teorie obvodů	1
Haničinec	Matematika 1	1

- podle těchto dat to vypadá, že **student** → **známka**
to ale ve skutečnosti určitě obecně neplatí, je to pouze náhoda

Normální formy

- aby schema relace splňovalo jisté požadavky (např. na malou redundanci, odstranění aktualizací anomálií atd.), zavádíme tzv. **normální formy**

- první
- druhá
- třetí
- Boyce-Coddova
- (čtvrtá, pátá)

První normální forma (1NF)

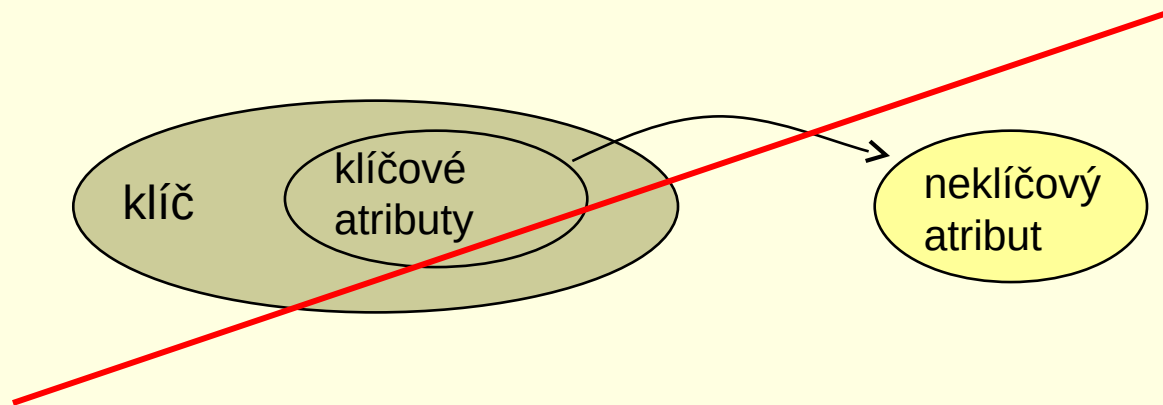
- každý atribut schématu je elementární a nestrukturovaný
 - tj. databáze je plochá, tabulka je skutečně dvojrozměrný objekt

První normální forma (1NF)

- každý atribut schématu je elementární a nestrukturovaný
 - tj. databáze je plochá, tabulka je skutečně dvojrozměrný objekt
 - OSOBA(jméno, rč, č_op) je v 1NF
 - OSOBA(jméno, rč, adresa(město, ulice, čp)) není v 1NF

Druhá normální forma (2NF)

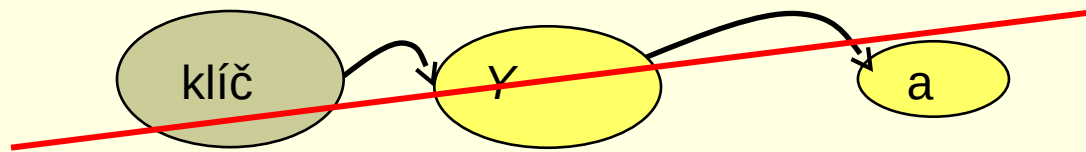
- neexistují závislosti atributů na podmnožině žádného klíče



- např. PROGRAM(název_k, jméno_f, adresa, datum)
není v 2NF, neboť platí, že $\text{název_k} \rightarrow \text{adresa}$

Třetí normální forma (3NF)

- nejdůležitější
- žádný neklíčový atribut není tranzitivně závislý na žádném klíči



- splněno, platí-li pro každou $X \rightarrow a$ (kde $X \subseteq A$, $a \in A$) aspoň jedna z podmínek
 - závislost je triviální
 - X je nadklíč
 - a je částí klíče

Třetí normální forma (3NF)

■ příklad

Firma	Sídlo	Kraj
Frantova firma	Praha	Praha
Martinova firma	Ostrava	Moravskoslezský
Pavlova firma	Brno	Jihomoravský
Viktorova firma	Praha	Praha
Pepova firma	Brno	Jihomoravský

Firma → *všechno*

Sídlo → Kraj

je ve 2NF, není ve 3NF (tranzitivní závislost Kraje na klíči přes Sídlo)

důsledek: redundance hodnot Kraje

Třetí normální forma (3NF)

■ příklad - řešení

Firma	Sídlo
Frantova firma	Praha
Martinova firma	Ostrava
Pavlova firma	Brno
Viktorova firma	Praha
Pepova firma	Brno

Firma → *všechno*

Sídlo	Kraj
Praha	Praha
Ostrava	Moravskoslezský
Brno	Jihomoravský

Sídlo → *všechno*

obě schémata jsou ve 3NF

Jak normalizovat

- mějme schema relace $R(\underline{A}, B, C)$, kde atribut A je klíčem
 - to, že je A klíčem znamená mj. že:
 - $A \rightarrow B$
 - $A \rightarrow C$
 - klíč totiž jednoznačně určuje každou n-tici relace,
 - každý atribut relace je na klíči přímo závislý
 - možná namítnete, že relace R má tři atributy a výše uvedené závislosti jsou jen dvě; ta třetí je ovšem triviální, proto jsem ji neuvedl: $A \rightarrow A$.
 - přijdeme-li při analýze na to, že na relaci R platí také $B \rightarrow C$, pak tato relace není ve 3NF:
 - existuje zde tranzitivní závislost atributu C na klíči přes atribut B

Jak normalizovat

■ řešení

- dekomponujeme původní relaci na dvě a to tak, že problematickou funkční závislost umístíme do samostatné relace, atribut pravé strany z původní relace odtrhneme:
 - $R1(\underline{B}, C)$ -- nová relace
 - $R2(\underline{A}, B)$ -- původní relace bez pravé strany problematické funkční závislosti
- o nic jsme přitom nepřišli, protože pokud provedeme nyní spojení ($R2 * R1$), dostaneme původní relaci R

Poznámky

- normalizace – výhody

- vede k lepšímu uložení dat v databázi
- omezuje redundanci
- snižuje riziko nekonzistence

- nevýhody

- zvyšuje četnost operace spojení, která je paměťově i časově náročná, proto někdy raději volíme nenormalizované relace kvůli rychlejší odezvě
- může značně znepřehlednit databázi
- někdy normalizací sice vzniknou relace ve 3NF, ale dojde ke ztrátě některých závislostí mezi daty, je třeba obezřetnosti